

Level-tracking LED

The external LED glows in proportion to the water level, and blinks once the alert line is crossed.

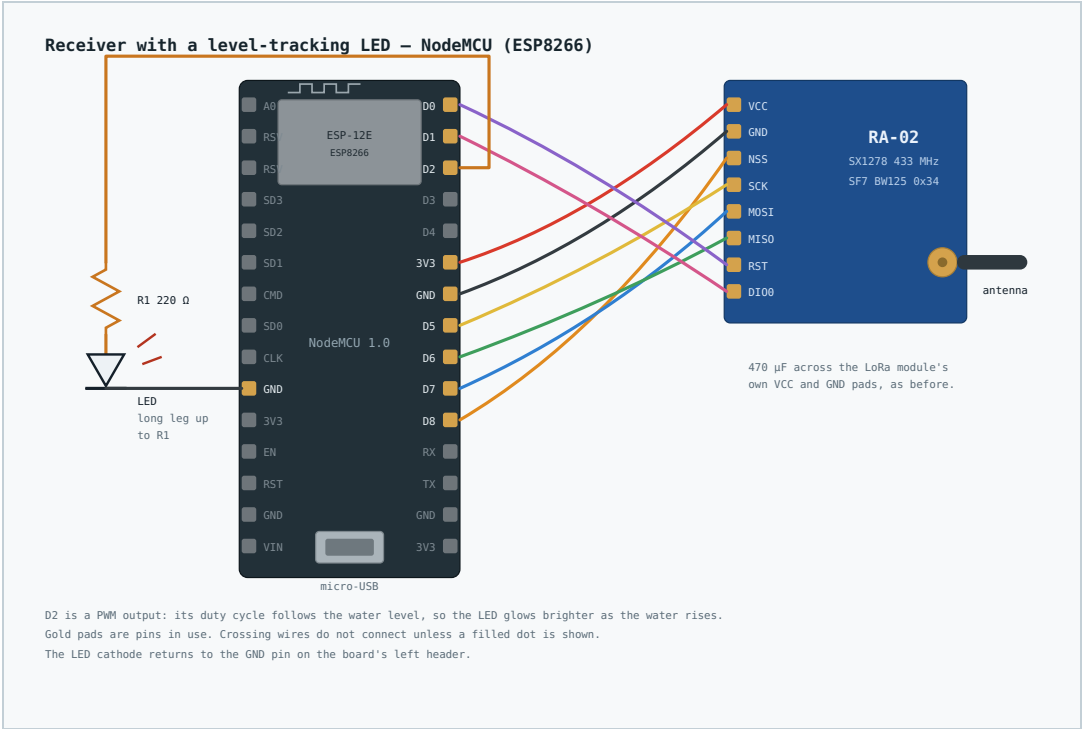


Fig. 1 Receiver wiring. Only one wire changes from the plain version: D2 now drives the LED through 220 Ω instead of 330 Ω , because a dimmable LED needs headroom at the bright end.

Wire list

NODEMCU PIN	GPIO	GOES TO	WIRE
3V3	–	LoRa VCC (+ 470 μ F at its pads)	red
GND	–	LoRa GND and the LED cathode	black
D8	15	LoRa NSS	orange
D5	14	LoRa SCK	yellow
D7	13	LoRa MOSI	blue
D6	12	LoRa MISO	green
D0	16	LoRa RST	purple
D1	5	LoRa DIO0	pink
D2	4	220 Ω → LED anode (PWM)	amber

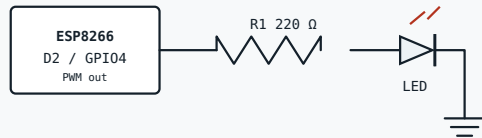
SCK, MISO AND MOSI ARE NOT YOURS TO MOVE

The ESP8266's hardware SPI is wired to D5, D6 and D7. Only NSS, RST and DIO0 can be reassigned. If the board refuses to boot, move NSS off D8 (GPIO15 must be low at power-up) to D4 / GPIO2 and change L0RA_SS to 2.

How the brightness is produced

D2 is a PWM output. The sketch sets its duty cycle from the level in every received packet — twice a second — so the glow follows the water continuously rather than in steps.

A · Direct drive – what the sketch expects

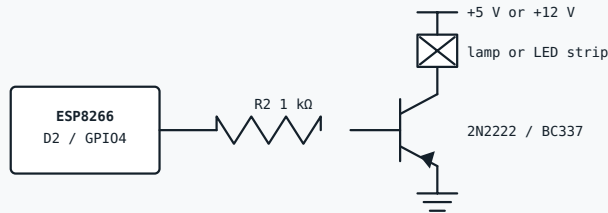


$I = (3.3\text{ V} - 2.0\text{ V}) / 220\ \Omega = 5.9\text{ mA}$ at full duty, well under the ESP8266's 12 mA per-pin limit.

330 Ω also works and gives 3.9 mA – dimmer, and the low end of the ramp becomes hard to see.

A blue or white LED drops about 3.0 V, so use 100 Ω with those or the glow never gets properly bright.

B · Transistor drive – for a lamp, a strip, or a high-power LED



The PWM signal still comes from D2, so the brightness behaviour is identical – the transistor only carries the current the pin cannot.

Grounds must be common. If you switch a relay or a motor instead, put a 1N4007 across it, cathode to the positive rail.

Fig. 2 Direct drive is what the sketch assumes. The transistor stage changes nothing in firmware; it only carries current the pin cannot.

What you will actually see

WATER LEVEL	GAP TO SENSOR	DUTY CYCLE	LED
0.0 ft	3.0 ft	0 %	dark
0.5 ft	2.5 ft	6 %	just visible
1.0 ft	2.0 ft	25 %	dim glow
1.5 ft	1.5 ft	56 %	clearly lit
2.0 ft	1.0 ft	100 %	full — and starts blinking
above 2.0 ft	under 1.0 ft	blink	4 Hz alarm

The duty column is not linear with level, deliberately. The eye responds to roughly the square root of light output, so a straight mapping looks dead for the first half of the climb and then leaps. Squaring the fraction cancels that, and the ramp looks even as the water rises.

The firmware that does it

```
#define PWM_MAX 1023 // the ESP8266's analogWrite range

void ledPwm(uint16_t v) {
  if (v > PWM_MAX) v = PWM_MAX;
  analogWrite(PIN_LED, v);
  analogWrite(PIN_LED_ONBOARD, PWM_MAX - v); // onboard is inverted
}

// The eye responds roughly to the square root of light output, so a
// straight level-to-duty mapping looks like it does nothing for the
// first half and then jumps. Squaring the fraction cancels that out.
uint16_t brightnessFor(int16_t level, int16_t depth) {
  if (level <= 0 || depth <= 0) return 0;
  float f = (float)level / (float)depth; // 0.0 empty, 1.0 at the sensor
  if (f > 1.0f) f = 1.0f;
  float alertF = (float)(depth - ALERT_GAP_MM) / (float)depth;
  if (alertF > 0.05f) f = f / alertF; // full brightness AT the alert line
  if (f > 1.0f) f = 1.0f;
  return (uint16_t)(f * f * PWM_MAX + 0.5f); // gamma 2.0
}

// in loop():
bool fresh = (goodCount > 0) && (now - lastHeard < 5000);
if (!fresh) { // no data - wink every 2 s
  if (now > beatNext) { beatNext = now + 2000; ledPwm(PWM_MAX); ledNext = now + 30; ledState = true; }
  else if (ledState && now > ledNext) { ledState = false; ledPwm(0); }
} else if (alert) { // above the line - blink
  if (now >= ledNext) { ledState = !ledState; ledPwm(ledState ? PWM_MAX : 0); ledNext = now + 125; }
} else { // normal - glow with the level
  ledPwm(brightnessFor(levelMM, depthMM));
}
```

THE BOOT FADE IS A TEST

At startup the sketch fades the LED up and down once. If you see that smooth fade, the PWM path is proven before any packet arrives — wiring, resistor, polarity and pin all confirmed in one second.

ON AN ESP32 OR ESP32-C3 THE PWM CALL IS DIFFERENT

Those chips use the LEDC peripheral, not analogWrite with a 0–1023 range. On ESP32 Arduino core 3.x the equivalent is `ledcAttach(PIN_LED, 1000, 10)` in setup and `ledcWrite(PIN_LED, v)` in place of `analogWrite`. Everything else, including the brightness curve, is unchanged.

